

1. **Modularny monolit** - Na start łatwe w implementacji, możliwość przejścia dalej w mikro-serwis
2. **Mikro-serwisy** - Na razie za trudne

1. Aplikacja do zarządzania przepływem pracy
2. Możliwość zarządzania użytkownikami - CRUD
3. Możliwość zarządzania rolami, uprawnień pracowników - CRUD
4. Możliwość zarządzania zadaniami/formularzami - CRUD

Jak to zrobić

Domeny

Co jest potrzebne do zrobienia tego?

1. Użytkownicy
  - a. Admin
  - b. Pracownik
  - c. Menadżer
2. Uprawnienia na poziomie zespołu/pracy
3. Zadania/Formularze

Wspólne:

1. Flow pracy/zadania

Narzędzia do implementacji:

1. Spring Boot
2. SQL
3. Auth0
4. AMQP -  
spring.boot.amqp.starter

Jak to zaimplementować?

Jak to zaimplementować?

Największe problemy:

1. Spring Security/JWT

Rozwiązania:

1. Użytkownik loguje się za pomocą email/hasło.  
Aplikacja komunikuje się z Auth0 aby otrzymać informacje na temat metadanych użytkownika, o ile istnieje. Jeżeli użytkownik nie istnieje zwracany jest odpowiedni błąd.
2. Użytkownik wysyła zapytanie z tokenem, który dostał od naszej aplikacji. Aplikacja konwertuje token do odpowiedniego modelu domenowego. Na podstawie modelu domenowego podejmuje akcje względem tego czy użytkownik ma uprawnienie do wykonania odpowiedniej akcji.
3. Auth0 będzie trzymane tylko i wyłącznie do trzymania danych autentykacji i autoryzacji użytkownika w to wchodzi role z domeny Uprawnienia

Ułożenie na poziomie projektu:

1. root (src)
  - a. commons
    - i. CommonsConfig.java
  - b. auth
    - i. WebSecurityConfig.java
    - ii. ....
  - c. user-module
    - i. config
      1. UserConfig.java
    - ii. repo
      1. UserRepo.java
    - iii. listeners
      1. UserListener.java
  - d. privilege-module
  - e. forms(tasks)-module

Wszystko musi być **KISS**

Kolejki:

1. Tworzysz Exchange do Queue (np. user\_exchange -> user\_queue)
2. Dodatkowo tworzysz DLQ (user\_exchange\_dlq -> user\_queue\_dlq -> user\_queue)